LESSON 20

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Wi-Fi Control Panel

Turn your ESP32 into a tiny website you can control from your phone.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How the ESP32 can make its **own Wi-Fi network**
- What an **IP address** is and how a browser finds a web page
- How to write a simple web page in **HTML**
- How a **web server** answers different addresses like `/on` and `/off`

SESSION PLAN

0:00	How does the internet find a web page?
0:10	Wire the LED and knob
0:20	Sketch 1: **L20_HelloWeb** — your first web page
0:50	Short break
0:55	Sketch 2: **L20_LedControl** — ON/OFF buttons
1:25	Challenges: design your control panel
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, 2 breadboards
- ☐ 1 LED + 220 Ω resistor
- ☐ Potentiometer (10 kΩ knob)
- ☐ Jumper wires
- ☐ A phone, tablet or laptop with Wi-Fi (one per group)

1. How does a web page reach you?

When you type a website into a browser, your device sends a message: "*please send me your page*". A computer called a **web server** answers by sending back the page, written in a language called **HTML**. Your browser draws it on screen.

Your ESP32 is powerful enough to be a web server! And it can even make its **own Wi-Fi network**, called an **access point**, so you don't need the school's Wi-Fi.

> **Big idea: IP addresses are like house numbers**
>
> Every device on a network has an **IP address**, like `192.168.4.1`. It's like a house number: it tells messages where to go. When your ESP32 makes its own network, it always gives itself the address **192.168.4.1**. Type that into a browser and you're knocking on your ESP32's door.

2. Wire it up

✓	From	To
☐	ESP32 pin **26**	220 Ω resistor → **long leg** of the LED
☐	LED **short leg**	**–** rail
☐	Knob **left** leg	**GND**
☐	Knob **middle** leg	ESP32 pin **34**
☐	Knob **right** leg	**3V3**

3. Sketch 1: Hello, web!

Type this in and save as **L20_HelloWeb**.

> **Pick a unique Wi-Fi name!**
>
> Everyone in the room is making a Wi-Fi network. Change `Sparky-ESP32` to your own name, like `Maya-ESP32`, so you can find yours. The password must be **at least 8 characters**.

> **A tiny bit of HTML**
>
> HTML uses **tags** in angle brackets. Most come in pairs: `<h1>` starts a big heading and `</h1>` ends it. `<p>…</p>` is a paragraph. Our sketch builds the page as a **String**, adding one piece at a time with `+=`.

Type this in → save it as **L20_HelloWeb** — 36 lines

```
// Lesson 20: Hello, web!
// The ESP32 makes its own Wi-Fi network and serves a web page.

#include <WiFi.h>
#include <WebServer.h>

String WIFI_NAME = "Sparky-ESP32";   // make this unique!
String PASSWORD = "secret123";       // at least 8 characters
const int POT_PIN = 34;

WebServer server(80);

void handleRoot() {
  String page = "<html><body style='font-family:sans-serif'>";
  page += "<h1>Hello from my ESP32!</h1>";
  page += "<p>Knob reading: ";
  page += analogRead(POT_PIN);
  page += "</p></body></html>";
  server.send(200, "text/html", page);
}

void setup() {
```

```
  Serial.begin(115200);
  WiFi.softAP(WIFI_NAME, PASSWORD);
  Serial.print("Join Wi-Fi ");
  Serial.print(WIFI_NAME);
  Serial.print(" then open http://");
  Serial.println(WiFi.softAPIP());

  server.on("/", handleRoot);
  server.begin();
}

void loop() {
  server.handleClient();
}
```

Now connect:

- [ ] Upload, and open the **Serial Monitor** (115200 baud). Press **EN** if nothing shows.
- [ ] On your phone, open **Wi-Fi settings** and join your network. Type your password.
- [ ] Your phone may say "*No internet*". That's OK, stay connected!
- [ ] Open a browser and type **192.168.4.1** in the address bar.
- [ ] Turn the knob and **refresh** the page. The number changes!

How it works

Code	What it means
`WiFi.softAP(WIFI_NAME, PASSWORD);`	Make a Wi-Fi network (a **soft Access Point**)
`WiFi.softAPIP()`	The ESP32's own IP address: 192.168.4.1
`WebServer server(80);`	A web server. Web pages use "door number" (**port**) 80
`server.on("/", handleRoot);`	When someone asks for the main page `/`, run `handleRoot()`
`server.send(200, "text/html", page);`	Send the page back. **200** means "OK, here it is"
`server.handleClient();`	Check for visitors. Must run all the time, so it lives in `loop()`

Try it

1. Change the heading to say something about you.
2. Add another line: `page += "<p>I am 12 years old</p>";` (or anything you like).
3. Ask a friend to join **your** network and visit your page. Does it work on two phones at once?

4. Sketch 2: Buttons that control the LED

Save as **L20_LedControl** and type this in. The Wi-Fi name and password stay the same as yours.

Type this in → save it as **L20_LedControl** 58 lines

```
// Lesson 20: Control an LED from your phone
// Buttons on the web page switch the LED on pin 26.

#include <WiFi.h>
#include <WebServer.h>

String WIFI_NAME = "Sparky-ESP32";
String PASSWORD = "secret123";
const int POT_PIN = 34;
const int LED_PIN = 26;

WebServer server(80);
bool ledOn = false;

void sendPage() {
  String page = "<html><head><meta name='viewport' ";
  page += "content='width=device-width'></head>";
  page += "<body style='font-family:sans-serif;text-align:center'>";
  page += "<h1>My ESP32</h1>";
  page += "<p>LED is ";
  if (ledOn) page += "ON";
  else page += "OFF";
  page += "</p><p>Knob: ";
  page += analogRead(POT_PIN);
  page += "</p>";
  page += "<a href='/on'><button>ON</button></a> ";
  page += "<a href='/off'><button>OFF</button></a>";
  page += "</body></html>";
  server.send(200, "text/html", page);
}

void handleOn() {
  ledOn = true;
  digitalWrite(LED_PIN, HIGH);
  sendPage();
}

void handleOff() {
  ledOn = false;
  digitalWrite(LED_PIN, LOW);
  sendPage();
}

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  WiFi.softAP(WIFI_NAME, PASSWORD);
```

```
  Serial.println(WiFi.softAPIP());

  server.on("/", sendPage);
  server.on("/on", handleOn);
  server.on("/off", handleOff);
  server.begin();
}

void loop() {
  server.handleClient();
}
```

Join your network again and go to **192.168.4.1**. Press **ON** and **OFF**. Watch the LED!

How it works

Code	What it means
`<a href='/on'><button>ON</button></a>`	A button that is also a **link**: pressing it visits the address `/on`
`server.on("/on", handleOn);`	When a browser visits `/on`, run `handleOn()`
`digitalWrite(LED_PIN, HIGH);` then `sendPage();`	Switch the LED on, then send the page back so the phone shows the new state
`<meta name='viewport' …>`	Tells phones to show the page at a sensible size
`'` inside `"…"`	HTML needs quotes too. We use single `'` quotes inside the String's double `"` quotes

Look at the address bar

After pressing ON, the address says `192.168.4.1/on`. Every web page you visit works this way: the part after the `/` tells the server *what* you want.

Page won't load?

Check your phone is still on the ESP32's network; some phones switch back to a network that has internet. Turning off **mobile data** can help.

5. Challenges

Challenge 1: Make it pretty

Add colour with **styles**. Try `<button style='font-size:30px;background:green;color:white'>`. Change the page background with `<body style='background:lightyellow'>`.

Challenge 2: Beep button

Add a buzzer on pin **33** and a **BEEP** button that visits `/beep` and plays a tone.

Challenge 3: Auto-refresh

Make the knob reading update by itself. Add this inside `<head>`: `<meta http-equiv='refresh' content='2; url=/'>`. What does the `2` do?

Bonus: Brightness levels

Make **LOW**, **MEDIUM** and **HIGH** buttons that use `analogWrite` to set the LED brightness. *Hint: you can visit `/dim?level=40` and read the number with `server.arg("level").toInt()`.*

6. Quiz

Check what you learned

1. What IP address does the ESP32 give itself in access-point mode?

2. What does `server.on("/off", handleOff);` mean, in your own words?

3. What is the difference between `<h1>` and `</h1>`?

4. Why should everyone in the class use a different Wi-Fi name?

Tidy up: unplug the USB cable, and on your phone choose **Forget this network** so it reconnects to your normal Wi-Fi.